

# **Отчёт о лабораторной работе**

**Лабораторная работа 7**

Андрюшин Никита Сергеевич

# Содержание

|          |                                       |           |
|----------|---------------------------------------|-----------|
| <b>1</b> | <b>Цель работы</b>                    | <b>5</b>  |
| <b>2</b> | <b>Выполнение лабораторной работы</b> | <b>6</b>  |
| <b>3</b> | <b>Выводы</b>                         | <b>14</b> |

# Список иллюстраций

|      |                                                                     |    |
|------|---------------------------------------------------------------------|----|
| 2.1  | Запуск сервера . . . . .                                            | 6  |
| 2.2  | Копирование конфигурационного файла . . . . .                       | 7  |
| 2.3  | Редактирование файла /etc/firewalld/services/ssh-custom.xml . . . . | 8  |
| 2.4  | Список служб firewalld . . . . .                                    | 8  |
| 2.5  | Списки служб . . . . .                                              | 9  |
| 2.6  | Добавление службы как активной . . . . .                            | 10 |
| 2.7  | Форвардинг портов . . . . .                                         | 10 |
| 2.8  | Подключение по ssh . . . . .                                        | 10 |
| 2.9  | Включение перенаправления и маскардинга . . . . .                   | 11 |
| 2.10 | Проверка доступа в интернет . . . . .                               | 12 |
| 2.11 | Сохранение конфигурации vagrant . . . . .                           | 12 |
| 2.12 | firewall.sh . . . . .                                               | 13 |
| 2.13 | Vagrantfile . . . . .                                               | 13 |

## **Список таблиц**

# 1 Цель работы

Получить навыки настройки межсетевого экрана в Linux в части переадресации портов и настройки Masquerading.

## 2 Выполнение лабораторной работы

Запустим наш сервер через vagrant (рис. 2.1).

```
C:\work\nsandryushin\vagrant>vagrant up server
Bringing machine 'server' up with 'virtualbox' provider...
==> server: You assigned a static IP ending in ".1" or ":1" to this machine.
==> server: This is very often used by the router and can cause the
==> server: network to not work properly. If the network doesn't work
==> server: properly, try changing this IP.
==> server: You assigned a static IP ending in ".1" or ":1" to this machine.
==> server: This is very often used by the router and can cause the
==> server: network to not work properly. If the network doesn't work
==> server: properly, try changing this IP.
==> server: Resuming suspended VM...
==> server: Booting VM...
==> server: Waiting for machine to boot. This may take a few minutes...
server: SSH address: 127.0.0.1:2222
server: SSH username: vagrant
server: SSH auth method: password
==> server: Machine booted and ready!
==> server: Machine already provisioned. Run `vagrant provision` or use the `--provision`
==> server: flag to force provisioning. Provisioners marked to run always will still run.
==> server: Running provisioner: common hostname (shell)...
```

Рис. 2.1: Запуск сервера

Теперь зайдём под суперпользователем и скопируем файл описания службы ssh, создав его копию с названием ssh-custom.xml (рис. 2.2).

```
[nsandryushin@server.nsandryushin.net ~]$ sudo -i
[sudo] password for nsandryushin:
[root@server.nsandryushin.net ~]# cp /usr/lib/firewalld/services/ssh.xml /etc/firewalld/se
rvices/ssh-custom.xml
[root@server.nsandryushin.net ~]# cd /etc/firewalld/services/
[root@server.nsandryushin.net services]# cat /etc/firewalld/services/ssh-custom.xml
<?xml version="1.0" encoding="utf-8"?>
<service>
  <short>SSH</short>
  <description>Secure Shell (SSH) is a protocol for logging into and executing commands on
remote machines. It provides secure encrypted communications. If you plan on accessing yo
ur machine remotely via SSH over a firewalled interface, enable this option. You need the
openssh-server package installed for this option to be useful.</description>
  <port protocol="tcp" port="22"/>
</service>
```

Рис. 2.2: Копирование конфигурационного файла

Теперь изменим его следующим образом. Основное отличие заключается в том, что порт для tcp назначен как 2022. Это XML-файл, который определяет пользовательскую службу для брандмауэра firewalld в Linux. Построчно опишем его:

Строка 1: `xml version="1.0" encoding="utf-8"`

Это стандартное объявление XML. Оно указывает, что файл использует версию XML 1.0 и кодировку символов UTF-8.

Строка 2: `service`

Это открывающий тег для корневого элемента. Он означает начало определения новой службы для firewalld.

Строка 3: `short SSH-New /short`

Этот тег задает короткое, удобное для чтения имя службы. В данном случае имя — “SSH-New”.

Строка 4: `description Long SSH description /description`

Здесь содержится более подробное описание службы. В этом примере используется текст “Long SSH description” (Длинное описание SSH).

Строка 5: `port protocol="tcp" port="2022"/`

Это ключевая строка. Она определяет, что данная служба использует протокол tcp и слушает порт 2022. Когда эта служба будет активирована в firewalld, брандмауэр

откроет порт 2022 для входящих TCP-соединений.

Строка 6: /service

Это закрывающий тег, который завершает определение службы. (рис. 2.3).

```
GNU nano 8.1 /etc/firewalld/services/ssh-custom.xml
<?xml version="1.0" encoding="utf-8"?>
<service>
  <short>SSH-New</short>
  <description>Long SSH description</description>
  <port protocol="tcp" port="2022"/>
</service>
```

Рис. 2.3: Редактирование файла /etc/firewalld/services/ssh-custom.xml

Выведем список доступных служб firewalld. Нашей созданной службы тут нет (рис. 2.4).

```
[root@server.nsandryushin.net services]# firewall-cmd --get-services
0-AD RH-Satellite-6 RH-Satellite-6-capsule afp alvr amanda-client amanda-k5-client amqp am
qps anno-1602 anno-1800 apcupsd aseqnet audit ausweisapp2 bacula bacula-client bareos-dire
ctor bareos-filedaemon bareos-storage bb bgp bitcoin bitcoin-rpc bitcoin-testnet bitcoin-t
estnet-rpc bittorrent-lsd ceph ceph-exporter ceph-mon cfengine checkmk-agent civilization-
iv civilization-v cockpit collectd condor-collector cratedb ctdb dds dds-multicast dds-uni
cast dhcp dhcpv6 dhcpv6-client distcc dns dns-over-quick dns-over-tls docker-registry docke
r-swarm dropbox-lansync elasticsearch etcd-client etcd-server factorio finger foreman fore
man-proxy freeipa-4 freeipa-ldap freeipa-ldaps freeipa-replication freeipa-trust ftp galler
a ganglia-client ganglia-master git gpsd grafana gre high-availability http http3 https id
ent imap imaps iperf2 iperf3 ipfs ipp ipp-client ipsec irc ircs iscsi-target isns jenkins
kadmin kdeconnect kerberos kibana klogin kpasswd kprop kshell kube-api kube-apiserver kube
-control-plane kube-control-plane-secure kube-controller-manager kube-controller-manager-s
ecure kube-nodeport-services kube-scheduler kube-scheduler-secure kube-worker kubelet kube
let-readonly kubelet-worker ldap ldaps libvirt libvirt-tls lightning-network llmnr llmnr-c
lient llmnr-tcp llmnr-udp managesieve matrix mdns memcache minecraft minidlna mndp mongodb
mosh mountd mpd mqtt mqtt-tls ms-wbt mssql murmur mysql nbd nebula need-for-speed-most-wa
nted netbios-ns netdata-dashboard nfs nfs3 nmea-0183 nrpe ntp nut opentelemetry openvpn ov
irt-imageio ovirt-storageconsole ovirt-vmconsole plex pmcd pmproxy pmwebapi pmwebapis pop3
pop3s postgresql privoxy prometheus prometheus-node-exporter proxy-dhcp ps2link ps3netshr
v ptp pulseaudio puppetmaster quassel radius radsec rdp redis redis-sentinel rootd rpc-bind
rquotad rsh rsyncd rtsp salt-master samba samba-client samba-dc sane settlers-history-col
lection sip sips slimevr slp smtp smtp-submission smtps snmp snmptls snmptls-trap snmptrap
spideroak-lansync spotify-sync squid sssd ssh statsrv steam-lan-transfer steam-streaming
stellaris stronghold-crusader stun stuns submission supertuxkart svdrp svn syncthing synct
hing-gui syncthing-relay synergy syscomlan syslog syslog-tls telnet tentacle terraria tftp
tile38 tinc tor-socks transmission-client turn turns upnp-client vdsms vnc-server vrrp war
pinator wbem-http wbem-https wireguard ws-discovery ws-discovery-client ws-discovery-host
ws-discovery-tcp ws-discovery-udp wsdd wsdd-http wsman wsmans xdmcp xmpp-bosh xmpp-client
xmpp-local xmpp-server zabbix-agent zabbix-java-gateway zabbix-server zabbix-trapper zabbix
x-web-service zero-k zerotier
```

Рис. 2.4: Список служб firewalld



Теперь перезапустим `firewalld` и снова выведем список доступных служб. Теперь наша служба отображается. Попробуем вывести список активных служб и увидим, что нашей службы пока в нём нет (рис. 2.5).

```
[root@server.nsandryushin.net services]# firewall-cmd --reload
success
[root@server.nsandryushin.net services]# firewall-cmd --get-services
0-AD RH-Satellite-6 RH-Satellite-6-capsule afp alvr amanda-client amanda-k5-client amqp amqps anno-1602 anno-1800 apcupsd aseqnet audit ausweisapp2 bacula bacula-client bareos-director bareos-filedaemon bareos-storage bb bgp bitcoin bitcoin-rpc bitcoin-testnet bitcoin-testnet-rpc bittorrent-lsd ceph ceph-exporter ceph-mon cfengine checkmk-agent civilization-iv civilization-v cockpit collectd condor-collector cratedb ctdb dds dds-multicast dds-unicast dhcp dhcpv6 dhcpv6-client distcc dns dns-over-quic dns-over-tls docker-registry docker-swarm dropbox-lansync elasticsearch etcd-client etcd-server factorio finger foreman foreman-proxy freeipa-4 freeipa-ldap freeipa-ldaps freeipa-replication freeipa-trust ftp galera ganglia-client ganglia-master git gpsd grafana gre high-availability http http3 https ident imap imaps iperf2 iperf3 ipfs ipp ipp-client ipsec irc ircs iscsi-target isns jenkins kadmin kdeconnect kerberos kibana klogin kpasswd kprop kshell kube-api kube-apiserver kube-control-plane kube-control-plane-secure kube-controller-manager kube-controller-manager-secure kube-nodeport-services kube-scheduler kube-scheduler-secure kube-worker kubelet kubelet-readonly kubelet-worker ldap ldaps libvirt libvirt-tls lightning-network llmnr llmnr-client llmnr-tcp llmnr-udp managesieve matrix mdns memcache minecraft minidlna mndp mongodb mosh mountd mpd mqtt mqtt-tls ms-wbt mssql murmur mysql nbd nebula need-for-speed-most-wanted netbios-ns netdata-dashboard nfs nfs3 nmea-0183 nrpe ntp nut opentelemetry openvpn ovirt-imageio ovirt-storageconsole ovirt-vmconsole plex pmcd pmproxy pmwebapi pmwebapis pop3 pop3s postgresql privoxy prometheus prometheus-node-exporter proxy-dhcp ps2link ps3netserver ptp pulseaudio puppetmaster quassel radius radsec rdp redis redis-sentinel rootd rpc-bind rquotad rsh rsyncd rtsp salt-master samba samba-client samba-dc sane settlers-history-collection sip sips slimevr slp smtp smtp-submission smtps snmp snmptls snmptls-trap snmptrap spideroak-lansync spotify-sync squid ssdp ssh ssh-custom statsrv steam-lan-transfer steam-streaming stellaris stronghold-crusader stun stuns submission supertuxkart svdrp svn syncthing syncthing-gui syncthing-relay synergy syscomlan syslog syslog-tls telnet tentacle terraria tftp tile38 tinc tor-socks transmission-client turn turns upnp-client vds vnc-server vrrp warpinator wbem-http wbem-https wireguard ws-discovery ws-discovery-client ws-discovery-host ws-discovery-tcp ws-discovery-udp wsdd wsdd-http wsman wsmans xdmcp xmpp-bosh xmpp-client xmpp-local xmpp-server zabbix-agent zabbix-java-gateway zabbix-server zabbix-trapper zabbix-web-service zero-k zerotier
[root@server.nsandryushin.net services]# firewall-cmd --list-services
cockpit dhcp dhcpv6-client dns http https ssh
```

Рис. 2.5: Списки служб

Запустим созданную нами службу `ssh-custom`. Убедимся, что она запущена и находится в списке активных служб, после чего добавим её как постоянную службу, и перезагрузим `firewalld` (рис. 2.6).

```
[root@server.nsandryushin.net services]# firewall-cmd --add-service=ssh-custom
success
[root@server.nsandryushin.net services]# firewall-cmd --list-services
cockpit dhcp dhcpv6-client dns http https ssh ssh-custom
[root@server.nsandryushin.net services]# firewall-cmd --add-service=ssh-custom --permanent
success
[root@server.nsandryushin.net services]# firewall-cmd --reload
success
```

Рис. 2.6: Добавление службы как активной

С помощью `firewalld` настроим форвардинг портов с 2022 на 22 (рис. 2.7).

```
[root@server.nsandryushin.net services]# firewall-cmd --add-forward-port=port=2022:proto=tcp:toport=22
success
```

Рис. 2.7: Форвардинг портов

Попробуем с клиента подключиться по `ssh` к серверу, используя именно 2022 порт. Как видим, операция прошла успешно (рис. 2.8).

```
[nsandryushin@client.nsandryushin.net ~]$ ssh -p 2022 nsandryushin@server.nsandryushin.net
The authenticity of host '[server.nsandryushin.net]:2022 ([192.168.1.1]:2022)' can't be established.
ED25519 key fingerprint is SHA256:1PtjeoTNJyZ4YiNPkJxzegCAqkN7GCRWznJJBakANJY.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '[server.nsandryushin.net]:2022' (ED25519) to the list of known hosts.
nsandryushin@server.nsandryushin.net's password:
Web console: https://server.nsandryushin.net:9090/ or https://10.0.2.15:9090/

Last login: Thu Oct 16 17:05:55 2025
[nsandryushin@server.nsandryushin.net ~]$
```

Рис. 2.8: Подключение по `ssh`

Проверим, включена ли в ядре опция перенаправления `ipv4` пакетов. Как видим, переключатель стоит в положении 0. Запишем настройку `net.ipv4.ip_forward = 1` в файл конфигурации `/etc/sysctl.d/90-forward.conf`. После этого загружаем этот конфигурационный файл и включаем через `firewall-cmd` маскардинг (рис. 2.9).

```
[root@server.nsandryushin.net services]# sysctl -a | grep forward
net.ipv4.conf.all.bc_forwarding = 0
net.ipv4.conf.all.forwarding = 0
net.ipv4.conf.all.mc_forwarding = 0
net.ipv4.conf.default.bc_forwarding = 0
net.ipv4.conf.default.forwarding = 0
net.ipv4.conf.default.mc_forwarding = 0
net.ipv4.conf.eth0.bc_forwarding = 0
net.ipv4.conf.eth0.forwarding = 0
net.ipv4.conf.eth0.mc_forwarding = 0
net.ipv4.conf.eth1.bc_forwarding = 0
net.ipv4.conf.eth1.forwarding = 0
net.ipv4.conf.eth1.mc_forwarding = 0
net.ipv4.conf.lo.bc_forwarding = 0
net.ipv4.conf.lo.forwarding = 0
net.ipv4.conf.lo.mc_forwarding = 0
net.ipv4.ip_forward = 0
net.ipv4.ip_forward_update_priority = 1
net.ipv4.ip_forward_use_pmtu = 0
net.ipv6.conf.all.forwarding = 0
net.ipv6.conf.all.mc_forwarding = 0
net.ipv6.conf.default.forwarding = 0
net.ipv6.conf.default.mc_forwarding = 0
net.ipv6.conf.eth0.forwarding = 0
net.ipv6.conf.eth0.mc_forwarding = 0
net.ipv6.conf.eth1.forwarding = 0
net.ipv6.conf.eth1.mc_forwarding = 0
net.ipv6.conf.lo.forwarding = 0
net.ipv6.conf.lo.mc_forwarding = 0
[root@server.nsandryushin.net services]# echo "net.ipv4.ip_forward = 1" > /etc/sysctl.d/90-
-forward.conf
[root@server.nsandryushin.net services]# sysctl -p /etc/sysctl.d/90-forward.conf
net.ipv4.ip_forward = 1
[root@server.nsandryushin.net services]# firewall-cmd --zone=public --add-masquerade --per
manent
success
[root@server.nsandryushin.net services]# firewall-cmd --reload
success
```

Рис. 2.9: Включение перенаправления и маскарadingа

Теперь с клиента попробуем зайти в сеть интернет. Как видим, интернет рабо-  
тает (рис. 2.10).

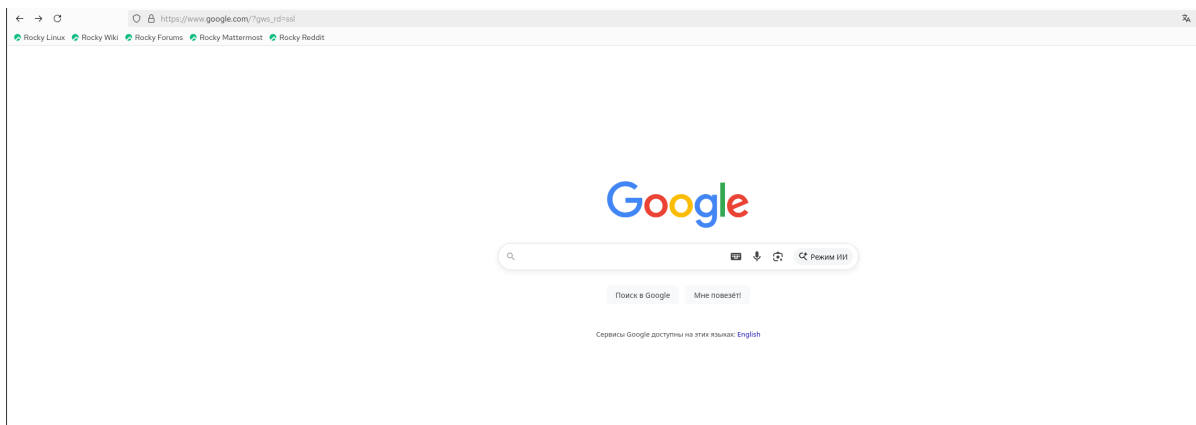


Рис. 2.10: Проверка доступа в интернет

После этого сохраним все конфигурационные файлы в vagrant и создадим скрипт firewall.sh (рис. 2.11).

```
[root@server.nsandryushin.net services]# cd /vagrant/provision/server
[root@server.nsandryushin.net server]# mkdir -p /vagrant/provision/server/firewall/etc/firewalld/services
[root@server.nsandryushin.net server]# mkdir -p /vagrant/provision/server/firewall/etc/sysctl.d
[root@server.nsandryushin.net server]# cp -r /etc/firewalld/services/ssh-custom.xml /vagrant/provision/server/firewall/etc/firewalld/services/
[root@server.nsandryushin.net server]# cp -r /etc/sysctl.d/90-forward.conf /vagrant/provision/server/firewall/etc/sysctl.d/
[root@server.nsandryushin.net server]# cd /vagrant/provision/server
[root@server.nsandryushin.net server]# touch firewall.sh
[root@server.nsandryushin.net server]# chmod +x firewall.sh
[root@server.nsandryushin.net server]# nano firewall.sh
[root@server.nsandryushin.net server]# █
```

Рис. 2.11: Сохранение конфигурации vagrant

В созданном скрипте firewall.sh пропишем следующие строки для настройки ssh и маскардинга (рис. 2.12).

```

GNU nano 8.1                               firewall.sh
#!/bin/bash
echo "Provisioning script $0"
echo "Copy configuration files"
cp -R /vagrant/provision/server/firewall/etc/* /etc
echo "Configure masquerading"
firewall-cmd --add-service=ssh-custom --permanent
firewall-cmd --add-forward-port=port=2022:proto=tcp:toport=22 --permanent
firewall-cmd --zone=public --add-masquerade --permanent
firewall-cmd --reload
restorecon -vR /etc

```

Рис. 2.12: firewall.sh

И запишем в vagrantfile автозагрузку этого скрипта (рис. 2.13).

```

89     server.vm.provision "server http",
90         type: "shell",
91         preserve_order: true,
92         path: "provision/server/http.sh"
93     server.vm.provision "server mysql",
94         type: "shell",
95         preserve_order: true,
96         path: "provision/server/mysql.sh"
97     server.vm.provision "server firewall",
98         type: "shell",
99         preserve_order: true,
100         path: "provision/server/firewall.sh"
101

```

Рис. 2.13: Vagrantfile

## **3 Выводы**

В результате выполнения лабораторной работы были получены навыки работы с фаерволом и настройкой форвардинга и маскардинга